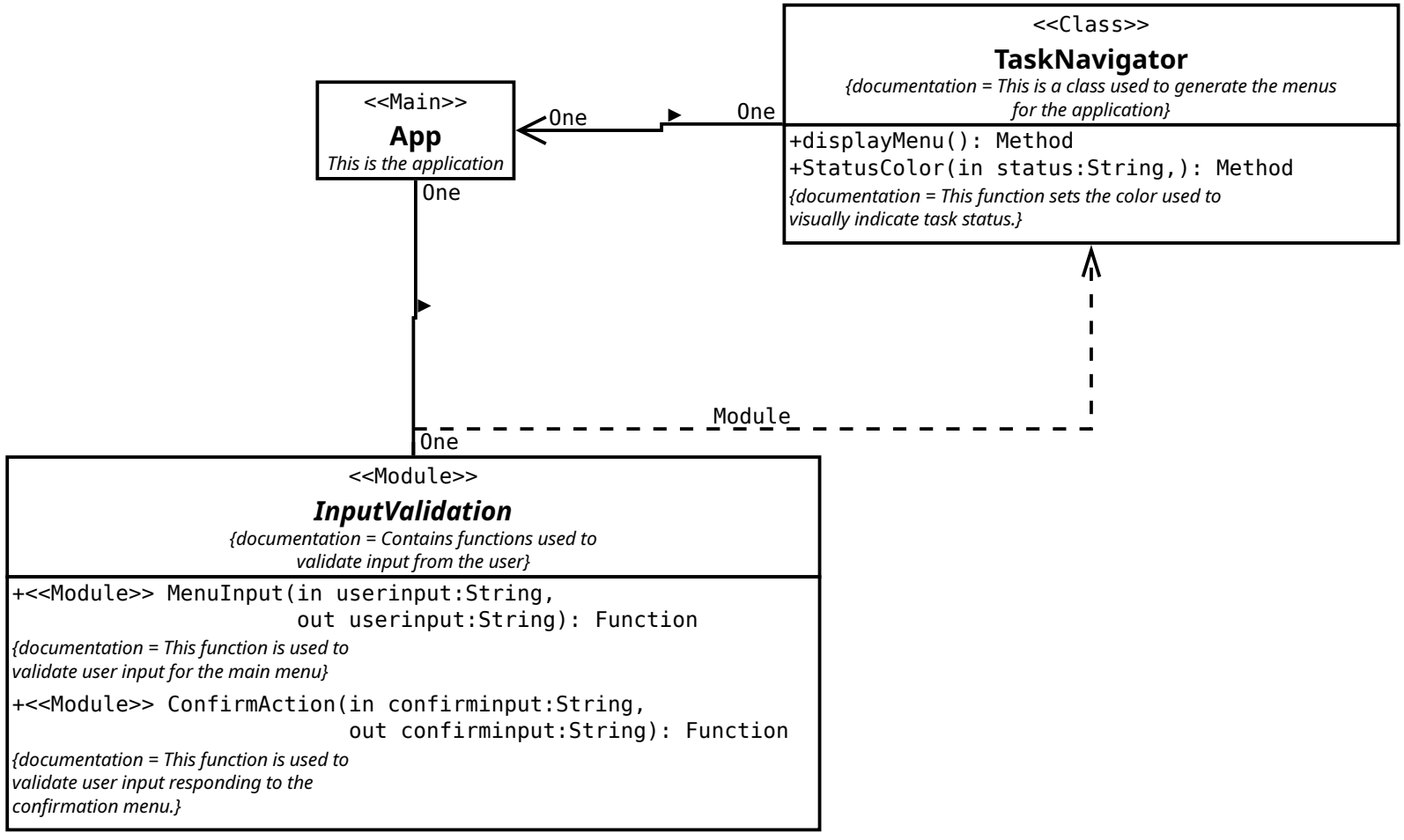

<<Class>>
TaskNavigator
{documentation = This is a class used to generate the menus for the application}
+displayMenu(): Method
+StatusColor(in status:String,): Method
{documentation = This function sets the color used to visually indicate task status.}
<<Main>>
App
This is the application
One
One
One
Module
One
<<Module>>
InputValidation
{documentation = Contains functions used to validate input from the user}
+<<Module>> MenuInput(in userinput:String, out userinput:String): Function
{documentation = This function is used to validate user input for the main menu}
+<<Module>> ConfirmAction(in confirminput:String, out confirminput:String): Function
{documentation = This function is used to validate user input responding to the confirmation menu.}

<<Main>>

**App**

*This is the application*

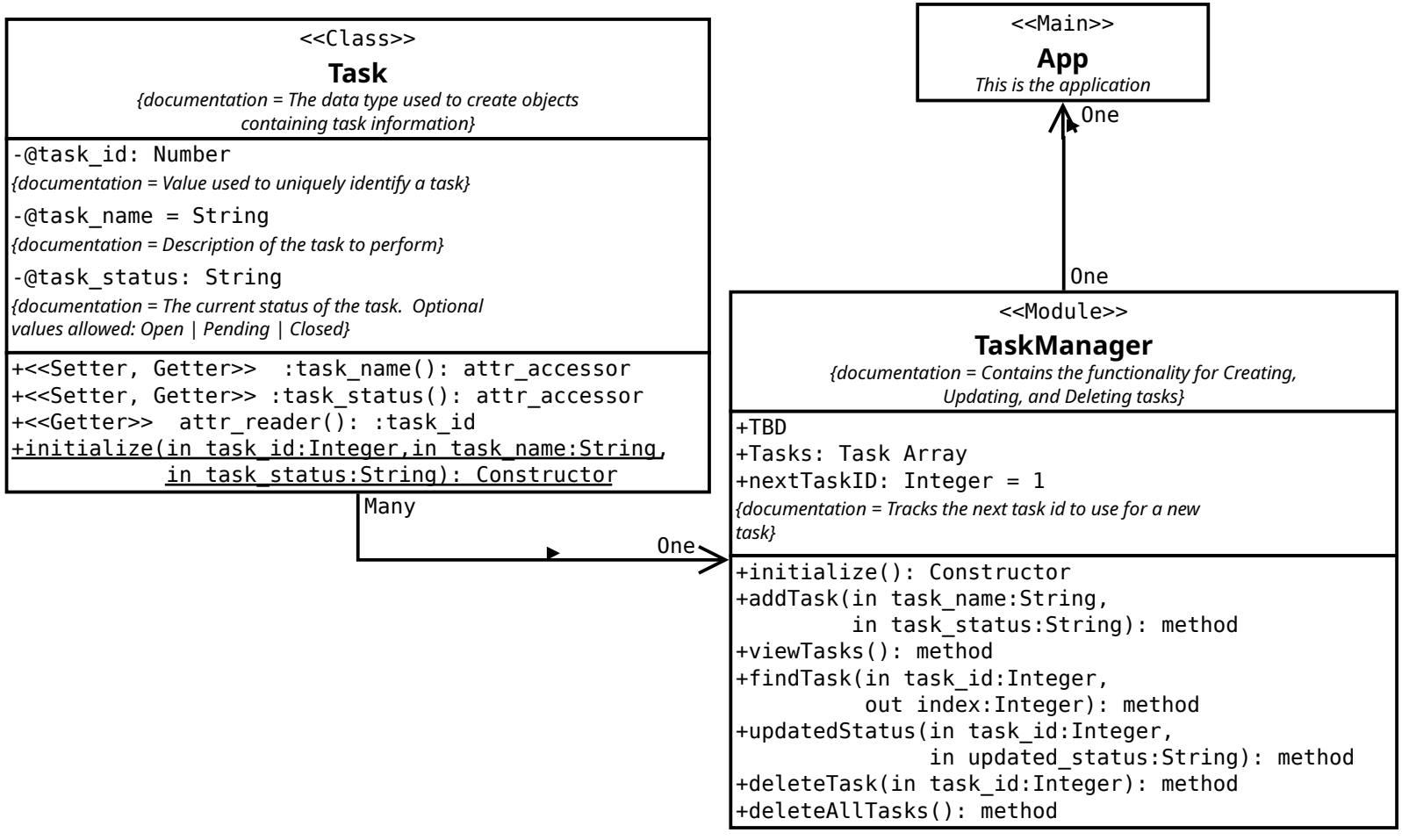
+crudmanager: TaskManager

*This is the main data structure used to manage user tasks*

+navigator: TaskNavigator

*This is the data structure that manages the menus*

+InputValidation.MenuInput(in userinput:String,out validatedInput:String)
+InputValidation.ConfirmTaskStatus(in confirmSelection:String,out validateSelection:String)
+InputValidation.ConfirmAction(in confirmAction:String,out validateAction:String)


<<Class>>
Task
{documentation = The data type used to create objects containing task information}
-@task_id: Number
{documentation = Value used to uniquely identify a task}
-@task_name = String
{documentation = Description of the task to perform}
-@task_status: String
{documentation = The current status of the task. Optional values allowed: Open | Pending | Closed}
+<<Setter, Getter>> :task_name(): attr_accessor
+<<Setter, Getter>> :task_status(): attr_accessor
+<<Getter>> attr_reader(): :task_id
+initialize(in task_id:Integer,in task_name:String, in task_status:String): Constructor
Many
One
<<Main>>
App
This is the application
One
One
<<Module>>
TaskManager
{documentation = Contains the functionality for Creating, Updating, and Deleting tasks}
+TBD
+Tasks: Task Array
+nextTaskID: Integer = 1
{documentation = Tracks the next task id to use for a new task}
+initialize(): Constructor
+addTask(in task_name:String, in task_status:String): method
+viewTasks(): method
+findTask(in task_id:Integer, out index:Integer): method
+updatedStatus(in task_id:Integer, in updated_status:String): method
+deleteTask(in task_id:Integer): method
+deleteAllTasks(): method